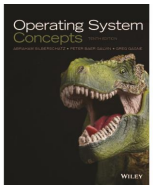



# Operating Systems 2026

**U3680**

**National Taipei University**

---

**Midterm Exam [Chapter 1 - Chapter 5]**

---

1. Answer the questions below about **interrupts**:

   (a) Describe why modern computers need interrupts to communicate with devices. (6%)
   Interrupts are essential for bridging the speed gap between high-speed CPUs and slower external devices. By replacing the inefficient polling practice, where the CPU wastes resources manually checking for data readiness, interrupts allow the system to operate asynchronously. This ensures the CPU remains focused on active tasks until an external signal triggers a necessary change in its execution flow.

   (b) Describe how modern systems reduce interrupt overheads when moving large files. (4%)
   Modern systems use Direct Memory Access (DMA). With DMA, an interrupt is triggered only after the entire block of data has been transferred, rather than for each byte, significantly reducing the number of context switches.

   (c) When is interrupt-based I/O not the best choice? Suggest a different method and explain how it works. (4%)
   In scenarios with extremely high data rates (e.g., high-speed network cards), frequent interrupts can lead to excessive overhead. In this case, polling is a more effective alternative because the CPU can process data in batches at its own pace.

2. CPU processing is fast, but memory access is slow. **Caches** are used to speed up data access.

   (a) Briefly explain how a cache works. (6%)
   Caches operate based on the principle of **locality**. When the CPU requests data, it first looks in the cache (a cache hit). If the data is found, it avoids the slower access to main memory. If the data is not in the cache, it is fetched from main memory and simultaneously stored in the faster cache closer to the CPU for quicker access in future requests.

   (b) Can a cache sometimes fail to improve performance? Explain your answer. (4%)
   Yes. If a program's memory access pattern is random or has poor locality, the Cache Miss Rate will be very high. In such cases, the cache adds overhead because the system must check the cache before eventually fetching data from main memory.

3. Describe two problems with **batch systems**. How do later computer systems address these two problems? (8%)
   The evolution from batch systems addressed two primary limitations: efficiency and interactivity. While batch systems often left the CPU idle by running only a single task, multi-programming enabled storing multiple jobs in memory, keeping the processor active. To bridge the gap in user engagement, time-sharing systems emerged, using rapid CPU rotations to provide an interactive environment in which users can manage tasks as they execute.

4. List **two** major drawbacks (or challenges) of using a **layered architecture** for OS design. (6%)

   - **Performance Overhead (slow):** Each layer adds a level of abstraction. A system call must pass through multiple layers, increasing the execution time and reducing overall throughput.
   - **Difficulty of Layer Definition:** It is difficult to identify which functions belong to each layer, and equally challenging to establish a global ordering.

5. Explain why modern operating systems provide APIs that encapsulate system calls, particularly focusing on ease of programming and portability. (8%)
   System calls are generally written in assembly language and designed to perform a single, specific task, whereas APIs are typically developed in high-level programming languages.

   - **Ease of Programming:** APIs provide high-level functions with intuitive parameters, enabling them to either combine multiple system calls into a single operation or avoid system calls entirely for routine tasks like mathematical computations.
   - **Portability:** Software built with standard APIs can be compiled and executed across different operating systems or hardware platforms without needing to modify the core logic. This is because compilers handle the underlying machine-specific complexities, such as instruction sets and register management.

6. Operating systems use short-term, medium-term, and long-term schedulers. **For each type**, identify which process state transitions it manages (your answers should be in the form of: from X state to Y state). (6%)

   - **Short-term:** from ready state to running state
   - **Medium-term:** from ready state to waiting state
   - **Long-term:** from new state to ready state

7. In a multi-threaded process, why must each thread have its own **stack** and **register set**? (6%)
   Each thread can execute a different portion of the code. It maintains its own register set, which holds the local computational state, and a stack that stores local variables and function call history; therefore, these cannot be shared.

8. Suppose 60% of a program's execution time is parallelizable. Can this program achieve a speedup of 2× or more on a **5-core** processor? Justify your answer with calculations. (8%)
   No, according to Amdahl's Law, the maximum speedup is:

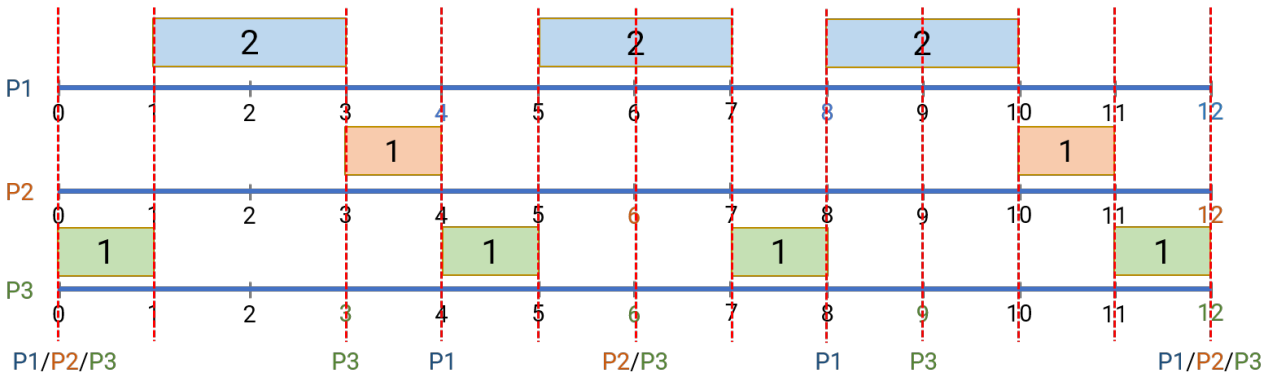
$$\frac{1}{0.4 + 0.6/5} = \frac{1}{0.52} < 2. \tag{1}$$

9. Load balancing uses both **push** and **pull** migration. Explain when an OS would use **push** and when it would use **pull** to balance the load. (6%)
   The system selects either push or pull migration based on CPU availability. When most CPUs are busy, pull migration is more effective. Conversely, when most CPUs are idle, push migration is the better option.

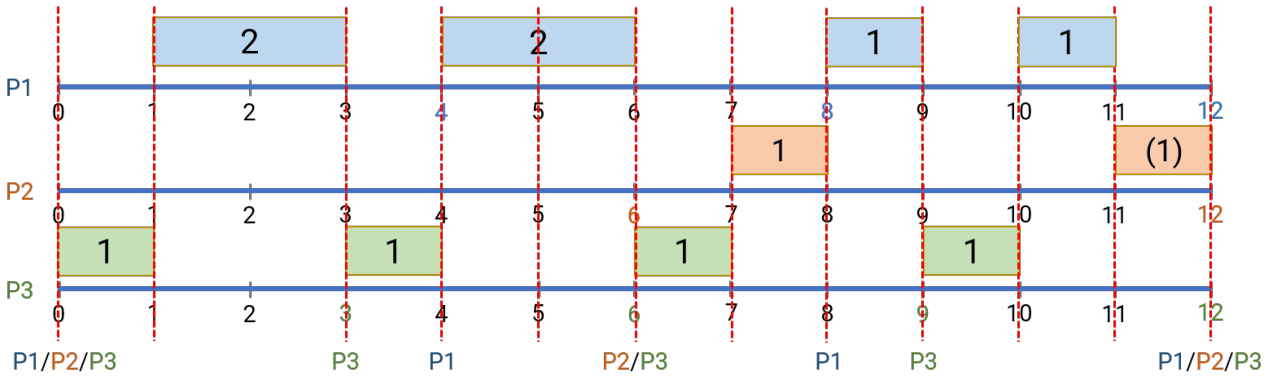
10. The following real-time processes are being scheduled using the **real-time scheduling algorithms**. Please show the scheduling order for the first 12 time units in a Gantt chart and **indicate whether any processes miss their deadlines. Be sure to label the specific time units when a process switch occurs**.

| Process | Ready | Execution | Period |
|---|---|---|---|
| $P_1$ | 0 | 2 | 4 |
| $P_2$ | 0 | 1 | 6 |
| $P_3$ | 0 | 1 | 3 |

(a) Use the **Early Deadline First (EDF)** algorithm to schedule these processes. If deadlines are tied, use priority: $P_1 > P_2 > P_3$. (6%)
There are no processes that miss their deadlines.



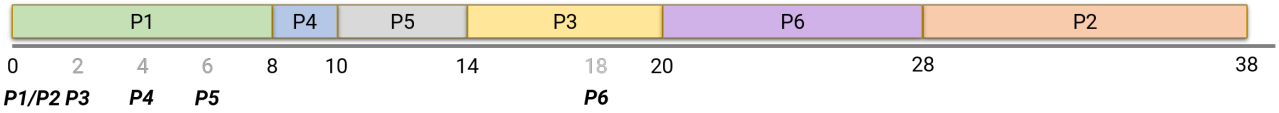

(b) Use the **Rate-Monotonic (RM)** algorithm to schedule these processes. (4%)
$P_2$ missed its deadline at $t = 6$.




11. The following processes are being scheduled using a **non-preemptive**, **shortest-job-first** scheduling algorithm:

| Process | Arrival | Burst |
|---|---|---|
| $P_1$ | 0 | 8 |
| $P_2$ | 0 | 10 |
| $P_3$ | 2 | 6 |
| $P_4$ | 4 | 2 |
| $P_5$ | 6 | 4 |
| $P_6$ | 18 | 8 |

(a) Show the scheduling order of the processes using a Gantt chart. (6%)




(b) What is the **waiting time** for each process? (6%)
$P_1$: 0; $P_2$: 28; $P_3$: 12; $P_4$: 4; $P_5$: 4; $P_6$: 2;

12. What are the outputs of the program shown below, assuming the 'sleep' program just does nothing and sleeps for one second? (6%)

```
#include <sys/types.h>
…

#define SIZE 3
int data[SIZE] = {1, 2, 3}

int main() {
    pid_t pid;
    pid = fork();
    if (pid > 0) {
        wait(NULL);
        for (int i = 0 ; i < SIZE ; ++i)
            printf("%d ", data[i]);
        printf("\n");
    }
    else if (pid == 0) {
        execlp("sleep", "sleep", NULL, NULL);
        for (int i = 0 ; i < SIZE ; ++i) {
            data[i] *= 2;
            printf("%d ", data[i]);
        }
        printf("\n");
        printf("child complete");
    }
    printf("program finished\n");
    return 0;
}
```

1 2 3
program finished